THM TECHNISCHE HOCHSCHULE MITTELHESSEN | CAMPUS FRIEDBERG | MND Mathematik, Naturwissenschaften und Datenverarbeitung

Skript zum Vorlesungsblock

Dokument-Datenbanken

Erstellt von: Marek Heimrich

Abgabe: 05.06.2023

Inhaltsverzeichnis

NoSQL 1
Was ist NoSQL? 1
Wozu NoSQL gebraucht wird 3
Dokument-Datenbanken 4
Was sind Dokument-Datenbanken? 4
Dokumenten-Datenbanken - Theoretische Grundlagen 4
JSON 4
BSON 6
BSON und Datumsangaben 6
MongoDB 7
Einführung 7
Collections 7
Dokumente 8
Insert 8
Update 9
Remove 10
Spezielle Operationen - Upsert 10
Indizes 10
Eingebettete Dokumente 11
Queries und Projektionen 11
Literatur 14

NoSQL

Was ist NoSQL?

NoSQL – ausgesprochen *Not only SQL* – steht für Datenbank-Ansätze abseits *relationaler Datenbanken*, welche üblicherweise die Datenbanksprache *SQL* verwenden. Verschiedene Datenbanksysteme werden zu den NoSQL-Datenbanken gezählt. Darunter beispielsweise XML-Datenbanken, Graph-Datenbanken oder auch Dokumenten-Datenbanken. Gemeinsam haben diese Systeme insbesondere ihre Flexibilität in Hinsicht auf die Datenspeicherung, welche in einem Gegensatz zu den relationalen Datenbanken steht. Konkret bedeutet dies, dass die Datenmodelle der NoSQL-Datenbanken i. d. R. weniger komplex ist als das, relationaler Datenbanken. Zu einem wird keine strenge Typisierung vorgenommen und Anforderungen an die Integrität der Datenbanken werden aufgeweicht.[1]

Exkurs: Relationale Datenbanken

Um die Abgrenzung von NoSQL zu relationalen Datenbanken (SQL-Datenbanken) besser greifbar zu machen, rufen wir uns hier kurz die wesentlichen Aspekte von relationalen Datenbanken ins Gedächtnis.

Relationale Datenbanksysteme (kurz: *RDMBS*) speichern Daten in einem relationalen Modell und verknüpfen verschiedene Datenpunkte miteinander (Relationen). Die Speicherung oder genauer gesagt die Darstellung der gespeicherten Daten erfolgt in Tabellenform – wir können uns dies also ähnlich vorstellen wie eine bzw. mehrere Tabelle(n) in Excel. Die Speicherung der Daten erfolgt dabei zeilenweise (*Tupel*). Jeder Datensatz, respektive Tupel, enthält einen Schlüsselwert, welcher eine eindeutige Identifizierung zulässt und zudem weitere Datenfelder, sog. Attribute. Beziehungen zwischen Tupeln aus verschiedenen Datenbanktabellen (*Relationen*) lassen sich über deren eindeutige Schlüsselwerte abbilden.[2]

Relationale Datenbanken werden weitläufig vor allem zur Erfassung von Geschäftsdaten (bspw. Transaktionsdaten) verwendet, beispielsweise für die Erfassung und Speicherung von Bestelldaten oder Kundenstammdaten. Relationale Datenbanken sind vor allem auf eine starke Konsistenz ausgelegt. D. h. die Datenbestände sollen immer aktuell sein, auch wenn mehrere Datenbank-Instanzen auf dieselben Datenbestände zugreifen. Somit bieten sich relationale Datenbanken sehr gut für Transaktions- und geschäftskritische Daten an. Besonders die Konsistenzanforderungen grenzen relationale Datenbanken von NoSQL-Datenbanken ab. Was dies konkret bedeutet, erfahren wir später.[3]

Schließlich gelten für relationale Datenbanken die sogenannten *ACID*-Eigenschaften. Die Abkürzung steht für die vier Eigenschaften *Atomarität*, *Konsistenz*, *Isolation* und *Dauerhaftigkeit.* Wir werden am Ende dieses Abschnitts erfahren, inwiefern sich, insbesondere in Bezug auf solche Eigenschaften, die NoSQL-Datenbanken bzw. im speziellen die Dokumentendatenbanken von relationalen Datenbanken abgrenzen.[4]

---

[1] Vgl. Saake, Sattler, Heuer, 2018, S. 661. Die Referenz bezieht sich auf den ganzen Absatz.
[2] Vgl. Oracle (o. A.), 2023a. Die Referenz bezieht sich auf den ganzen Absatz.
[3] Vgl. ebda. Die Referenz bezieht sich auf den ganzen Absatz.
[4] Vgl. ebda. Die Referenz bezieht sich auf den ganzen Absatz.

Nachdem wir uns im Exkurs das relationale Datenmodell noch einmal bewusst gemacht haben, werden wir nun das Datenmodell der NoSQL-Datenbanken genauer betrachten. Zunächst ist festzuhalten, dass NoSQL-Datenbanken nicht-relational sind. Ein Datenbankschema bzw. Datenmodell muss, im Gegensatz zu relationalen Datenbanken, nicht im Vorfeld definiert werden. Stattdessen kann dieses während des Anwendungsbetriebs definiert und auch laufend verändert werden. NoSQL-Datenbanken lassen sich somit sehr schnell in Betrieb nehmen. Zudem ermöglichen sie eine flexible und horizontale Skalierbarkeit, womit eine gute Anpassbarkeit an verschiedene Anforderungen möglich wird.[5]

Die horizontale Skalierbarkeit der NoSQL-Datenbanken stellt einen Gegensatz zu der meist vertikalen Skalierbarkeit relationaler Datenbanken dar. Bei Letzteren erfolgt die Skalierung über eine Änderung der verfügbaren Rechenressourcen auf dem jeweiligen Rechner bzw. Server. NoSQL-Datenbanken nutzen hingegen das sog. *Sharding*, bei welchem zur Skalierung weitere Rechnerknoten hinzugefügt werden und somit die Rechenlast beispielsweise auf verschiedene Server verteilt wird. Grundsätzlich sind NoSQL-Datenbanken auf eine verteilte Architektur ausgelegt und erlauben auch die Verteilung des Datenspeichers selbst. Dies prädestiniert NoSQL-Datenbanken für den Einsatz in einem Umfeld großer und ständig wachsender Datenmengen, wie Big Data.[6]

Die von relationalen Datenbanken bekannten *ACID*-Eigenschaften sollen dafür sorgen, dass sich die Datenbank stets in einem konsistenten Zustand befindet. Bei Operationen/Manipulationen auf der Datenbank, wird diese also ausgehend von einem konsistenten Zustand in einen neuen konsistenten Zustand überführt. Anstelle der ACID-Eigenschaften verfolgen NoSQL-Datenbanken die sog. *BASE*-Eigenschaften, welche als weniger restriktive Konsistenzanforderungen verstanden werden können, als es die ACID-Eigenschaften sind, wie wir in Kürze sehen werden. Zunächst soll jedoch auf eine weitere Besonderheit eingegangen werden, welche für verteilte Systeme und damit i. d. R. auch für NoSQL-Datenbanken gilt.[7]

Sobald wir über verteilte Systeme sprechen, wird ein Blick auf das *CAP-Theorem* notwendig. Dieses besagt, dass ein trade-off zwischen den drei Anforderungen Konsistenz, Verfügbarkeit und Ausfalltoleranz besteht. Konkret besagt das CAP-Theorem, dass nie alle dieser Anforderungen gleichzeitig vollständig erreicht werden können. Die Literatur spricht hier allerdings von graduellen Größen, d. h. die Erfüllung der einzelnen Anforderungen bewegt sich zwischen überhaupt nicht und vollständig.[8]

In Respekt zu dem trade-off des CAP-Theorems gewährleisten NoSQL-Datenbanken keine restriktive Konsistenz nach des ACID-Eigenschaften, sondern eine abgeschwächte Form von Konsistenz. Dies bedeutet, dass die Herstellung von Konsistenz nicht oberste Priorität hat, sondern ein konsistenter Zustand ggf. erst mit einigem zeitlichen Abstand zu einem Schreibvorgang hergestellt wird. Dies führt uns zu den bereits zuvor erwähnten BASE-Eigenschaften für NoSQL-Datenbanken.[9]

---

[5] Vgl. Oracle (o. A.), 2023b. Die Referenz bezieht sich auf den ganzen Absatz.
[6] Vgl. ebda. Die Referenz bezieht sich auf den ganzen Absatz.
[7] Vgl. brueck, 2015. Die Referenz bezieht sich auf den ganzen Absatz.
[8] Vgl. Trelle, 2014, Kap. 1.4, S. xvii f. Die Referenz bezieht sich auf den ganzen Absatz.
[9] Vgl. Trelle, 2014, Kap. 1.5, S. xix. Die Referenz bezieht sich auf den ganzen Absatz.

BASE steht für **B**asically **A**vailable, **S**oft state, **E**ventual consistency und beschreibt die klassischerweise anzutreffenden Anforderungen im Umfeld von verteilten Systemen. Diese Eigenschaften lassen sich wie folgt beschreiben[10]:

- **Basically Available:** Eine Antwort wird grundsätzlich auf jede Datenbankanfrage gegeben. Dies bedeutet aber, dass die zurückgegeben Daten inkonsistent oder auch in einem schwebenden Status sein können. Weiterhin kann auch eine Fehlermeldung zurückgegeben werden.
- **Soft state:** Das Datenbanksystem befindet sich ein einem sogenannten „Soft state", d. h. der Zustand des Systems könnte jederzeit Änderungen unterliegen auch dann, wenn keine Änderungen in die Datenbank eingegeben werden.
- **Eventual consistency:** Das Datenbanksystem wird in einen konsistenten Zustand überführt, sobald keine neuen Eingaben erfolgen. Dennoch können weiterhin Daten eingegeben werden, auch wenn noch kein konsistenter Zustand erreicht ist. D. h. das System erwartet keinen konsistenten Zustand, bevor neue Eingaben erfolgen können.

Wozu NoSQL gebraucht wird

Immer dann, wenn große Datenmengen verarbeitet werden müssen und eine flexible Skalierbarkeit gefragt ist, bietet sich der Einsatz von NoSQL-Datenbanken an. Das flexible Datenmodell erlaubt es Änderungen vorzunehmen, um auf sich verändernde Umstände zu reagieren. Da NoSQL-Datenbanken i. d. R. de-normalisierte Daten halten, eigenen sie sich tendenziell besser für Anwendungsfälle bei denen flexible und verschiedene Datenstrukturen verarbeitet werden müssen. Sie eigenen sich nicht für normalisierte Daten, wie sie klassischerweise im Back-Office von Unternehmen zum Einsatz kommen[11].

Die Zunahme der vorhandenen Datenmengen, insbesondere durch unsere heutige Internetnutzung und Social Media, aber auch die vielfache Sammlung aus Auswertung von Daten führen unter dem Begriff *Big Data* zu veränderten Anforderungen an Datenbanksysteme. Daten aus dem Umfeld von *Big Data* lassen sich mit vier Eigenschaften beschreiben: es handelt sich um enorm große Datenmengen, Datenaktualisierungen erfolgen millisekündlich (Stromdaten), die Daten sind heterogen und verrauscht. Relationale Datenbanksysteme sind nicht auf die Verarbeitung solcher Daten ausgelegt, wodurch sich ein Bedarf an alternativen Speichersystemen ergibt. Diesem Bedarf können NoSQL-Systeme besser begegnen, da sie auf eine Verarbeitung der Daten auf skalierbaren Rechnerclustern ausgelegt sind und zum Nachteil von restriktiven Konsistenzanforderungen auch heterogene und variable Datenstrukturen zulassen.[12]

Wie eingangs erwähnt wurde, existieren sehr unterschiedliche Ansätze für Datenbanksysteme, welche sich unter dem Paradigma NoSQL zusammenfassen lassen. Konkrete Beispiele hierfür sind beispielsweise *Key-Value-Stores*, *Wide-Column-Stores*, *Dokumenten-Datenbanken* und *Graphdatenbanken*. Jeder dieser Ansätze weist spezifische Charakteristika auf und eignet sich somit auch für spezifische Anwendungsfälle.[13] Wir beschäftigen uns im Folgenden nun näher mit den sog. Dokumenten-Datenbanken.

---

[10] Vgl. Roe, 2012. Die Referenz bezieht auf den ganzen Absatz und die Aufzählung.
[11] Vgl. Oracle (o. A.), 2023b. Die Referenz bezieht sich auf den ganzen Absatz.
[12] Vgl. Saake, Sattler, Heuer, 2018, S. 662 ff. Die Referenz bezieht sich auf den ganzen Absatz.
[13] Vgl. Saake, Sattler, Heuer 2018, S. 666. Die Referenz bezieht sich auf den ganzen Absatz.

Dokument-Datenbanken

Was sind Dokument-Datenbanken?

Dokument-Datenbanken zählen zur Klasse der *NoSQL*-Datenbanken und weisen eine hohe Ähnlichkeit zu den *Key-Value*-Datenbanken auf. Sie nutzen dabei den Ansatz zu einem zunächst vorhandenen Schlüssel (*Key*) Daten (*Value*) zuzuordnen. Die zu den Schlüsseln zugeordneten Daten sind dabei sogenannte *Dokumente*, d. h. komplexe und strukturierte Daten. Diese Dokumente enthalten dabei ebenfalls eine variable Anzahl von Schlüssel-Wert-Paaren.[14]

Besonders relevant ist in diesem Zusammenhang die Schemalosigkeit der Dokumente: für die Dokumente müssen im Vorfeld keine Schemen definiert werden. Außerdem kann jedes Dokument innerhalb der Datenbank über ein eigenes Schema verfügen, welches sich von den Schemen der anderen Dokumente unterscheidet.[15]

Dazu, wie solche Dokumente und ihre Schemen im Detail aussehen, kommen wir im Abschnitt *Theoretische Grundlagen*. Hier soll zunächst die zuvor beschriebene Schemalosigkeit anhand eines Beispiels gezeigt werden. Wir nehmen dafür an, dass die Datenbank *Werkstatt* verschiedene Daten über das Werkzeuginventar eines Hobbyhandwerkers hält. Zwei der Datenbankeinträge, „Bohrmaschine“ und „Werkzeugkasten“, könnten bei einer Datenabfrage beispielsweise folgende Dokumente ausgeben:

Bohrmaschine:

```
{
 "_id": 1,
 "Typ": "Bohrmaschine",
 "Gewicht": "2 KG",
 "Material": "Steinbohrer"
}
```

Werkzeugkasten:

```
{
 "_id": 2,
 "Typ": "Werkzeugkasten",
 "Größe": "Klein",
 "Einzelteile": 20,
 "Farbe": "Blau"
}
```

Wir sehen also, dass obwohl beide Dokumente in derselben Datenbank gespeichert sind, jedes Dokument ein eigenes Schema bzw. eine eigene Datenstruktur aufweist. Die Anzahl der Wertefelder und auch deren Datentypen unterscheiden sich voneinander.[16]

Das für Dokumenten-Datenbanken mit am häufigsten eingesetzte Datenformat ist das sog. *JSON*-Format.[17] Im folgenden Abschnitt *Theoretische Grundlagen* werden wir deshalb mehr über JSON, sowie dessen Abwandlung *BSON* erfahren.

Dokumenten-Datenbanken - Theoretische Grundlagen

JSON

JSON – für JavaScript Object Notation – wurde als Format zum Austausch von Daten entwickelt, um einerseits für Menschen leicht lesbar und schreibbar zu sein. Andererseits sollte ein einfaches Parsen und Generieren durch Maschinen möglich sein. Als textbasiertes Format ist JSON Programmiersprachen-unabhängig und ermöglicht so einen effektiven Datenaustausch. In JSON gibt es zunächst zwei Strukturen und fünf Entitäten bzw. Datentypen.

[14] Vgl. Saake, Sattler, Heuer, 2018, S. 671. Die Referenz bezieht sich auf den ganzen Absatz.
[15] Vgl. ebda. Die Referenz bezieht sich auf den ganzen Absatz.
[16] Vgl. Trelle, 2014, Kap. 8.1, S. 6b. Die Referenz bezieht sich auf den ganzen Absatz.
[17] Vgl. Saake, Sattler, Heuer, 2018, S. 671.

Als Strukturen werden zunächst *Name/Wert-Paare* (ähnlich eines dictionary) und *Geordnete Listen* (ähnlich eines array) spezifiziert. Darüber hinaus werden die folgenden Datentypen spezifiziert[18]:

- **Object:** Menge von Name/Wert-Paaren, wobei diese ungeordnet sind
- **Array:** Liste von Werten, diese Liste ist jedoch geordnet
- **Value:** ein Wert kann unterschiedlichen Typs sein. Bspw. ein Objekt, Array, String, Zahl, Boolean oder Null. Verschachtelungen sind dabei möglich
- **String:** Besteht aus Unicode-Zeichen, die Zeichenabfolge kann dabei durchaus auch aus keinen Zeichen bestehen
- **Number:** Zahlen, dabei ist zu beachten, dass oktale und hexadezimale Zahlen nicht unterstützt werden

JSON-Dokumente sind objektorientiert und folgen einer genau spezifizierten Struktur. So beginnt und schließt ein JSON-Dokument grundsätzlich mit geschweiften Klammern { … }. Innerhalb dieser Klammern können Objekte und Datenfelder gesetzt werden. Datenfelder erhalten zunächst einen Namen. Nach einem Doppelpunkt wird dann der Feld-Wert eingegeben. Weiterhin ermöglicht das JSON-Format mit Hilfe von JSON-Nodes eine hierarchische Strukturierung. So ist beispielsweise jedes Datenfeld ein Knoten und damit Kind des Hauptobjektes. Ein Objekt innerhalb des Hauptobjektes, welches wiederum weitere Datenfelder beinhaltet ist dann Elternknoten dieser weiteren Datenfelder. Jeder Knoten besitzt dabei einen eindeutigen Pfad innerhalb des JSON-Dokuments.[19]

Wir betrachten nun die Syntaxvorgaben für die jeweiligen JSON-Datentypen anhand von Beispielen. Die Beispiele stammen von Oracle[20]:

- **Object:**

```
{
  "car" :   { "type" : "SUV" ,  "price" : "80000" ,  "color" ,  "black" }
}
```

- **Array:**

```
{ "cars" :   [
{
 "type" : "SUV",
 "price" : 80000,
 "color" : "black"
}

{
 "type" : "Roadster",
 "price" : 60000
 "color" : "cashmere"
}
] }
```

[18] Vgl. JSON.org (o. A.), o. J. Die Referenz bezieht sich auf den ganzen Absatz und die Aufzählung.
[19] Vgl. Inray Industriesoftware GmbH (o. A.), 2023. Die Referenz bezieht sich auf den ganzen Absatz.
[20] Vgl. Oracle (o. A.), 2023c. Die Referenz bezieht sich auf die ganze Aufzählung und die Beispiele.

- **Bool und Null:**

```
{ "status_pay" : true }


{ "example_null" : null }
```

- **String:**

```
{ "sentence" : "Hier steht ein Satz" }
```

- **Number:**

```
{
  "zahl_int" : 7490,
  "zahl_double" : 832.58
}
```

BSON

Wir werden uns im weiteren Verlauf dieses Themas intensiv mit *MongoDB* beschäftigen. Bevor wir MongoDB näher kennenlernen, beschäftigen wir uns jedoch zunächst mit dem Datenformat, welches der MongoDB Dokumentendatenbank zugrunde liegt. Während man sich bei der Entwicklung von MongoDB schnell auf JSON als Datenformat geeinigt hat, stieß man auf zwei Probleme, welche die Nutzbarkeit von JSON zur Datenspeicherung einschränkten. Zum einen sind die Datentypen, welche JSON unterstützt nicht ausreichend umfänglich. Es fehlt an einer Unterstützung für Datumsangaben und Binärdaten. Zum anderen fehlt es JSON-Objekten an einer festgelegten Länge, was zu Performance-Problemen führt.[21]

Dies führte zu der Entwicklung des BSON-Formats, welches auf JSON basiert. BSON – für Binary JSON – ist eine binäre Repräsentation des JSON-Formats, welches für die Nutzung als Datenbank-Format optimiert wurde. Neben der Ergänzung von weiteren Datentypen, erlaubt das BSON-Format eine performantere Traversion der Einträge. Obwohl MongoDB Daten intern im BSON-Format verarbeitet, können grundsätzlich Daten, welche in JSON abgebildet werden können durch die internen Treiber in MongoDB gespeichert und abgerufen werden.[22]

BSON und Datumsangaben

Datumsangaben werden in MongoDB mit Hilfe von Strings oder *Date*-Objekten realisiert. Bei Aufruf der Methode *Date()* erhalten wir eine Datumsangabe als String, bei Aufruf der Methode *new Date()* ein Date-Objekt. Wir können verschiedene Datumsformate übergeben und so neben einem Datum auch Zeitangaben hinzufügen. Wir betrachten hier lediglich Datumsangaben, welche im Format *YYYY-mm-dd* übergeben werden. Die Methode *new Date()* gibt dann das Datum als sog. *ISODate* zurück. Wir können beispielsweise ein Geburtsdatum auf die folgende Weise speichern[23]:

db.uebung.update({Familienname: “Mustermann“}, { $set: { geb: new Date(“2000-01-01“)} })

[21] Vgl. MongoDB (o. A.), 2023a. Die Referenz bezieht sich auf den ganzen Absatz.
[22] Vgl. ebda. Die Referenz bezieht sich auf den ganzen Absatz.
[23] Vgl. Ian, 2021. Die Referenz bezieht sich auf den ganzen Absatz und das Beispiel.

MongoDB

Einführung

MongoDB ist der Name einer Dokumenten-Datenbank, welche Daten im zuvor erläuterten BSON-Format persistiert. Demnach bleiben die Vorteile von JSON bzw. allgemein NoSQL-Datenbanken, beispielsweise flexible Schemata, erhalten. MongoDB setzt auf einer verteilten Architektur auf und ermöglicht somit gute eine horizontale Skalierbarkeit der Datenbank. Zudem wird mit Hilfe verschiedener Aspekte, wie beispielsweise effektiver Indizierung, eine gute Performance erreicht.[24]

MongoDB kennt zunächst sog. *Collections* und *Dokumente*. Dokumente werden dabei in den Collections zusammengefasst. Diese ähneln in gewisser Weise den Tabellen aus RDBMS und können entweder explizit oder automatisch angelegt werden. Dokumente repräsentieren die Datensätze einer MongoDB-Datenbank. Die Repräsentation erfolgt wie zuvor beschrieben im JSON- bzw. BSON-Format.[25]

Aufgrund der Schemalosigkeit bzw. Schemafreiheit, welche für Collections gilt, ist es nicht notwendig Namen und Datentypen für die einzelnen Datenfelder der „zukünftigen“ Dokumente zu definieren. Wichtig ist, dass die Schemafreiheit rein technisch zu betrachten ist und in Abgängigkeit der Anwendung und vorliegenden Daten ein Schema sinnvoll sein kann.[26]

Collections

Wir wollen nun implizit eine Collection mit dem Namen *uebung* anlegen, indem wir ein erstes Dokument *test* anlegen.[27] Wir erstellen jedoch zuerst Datenbank *uebung_fdbt* mit Hilfe des folgenden Befehls über die Mongo Shell[28]:

use uebung_fdbt

Zum impliziten Anlegen der Collection und unseres ersten Dokuments nutzen wir folgenden Befehl[29]:

db.uebung.insert({test: “Dies ist ein Test zum impliziten Anlegen einer Collection.“})

Wenn wir nun den Befehl db.uebung().countDocuments()[30] eingeben sollte uns der Wert 1 zurückgegeben werden.

Die interne Speicherung von Collections erfolgt als sog. *Extents*, wobei eine Collection durchaus über mehrere Extents verteilt sein kann. Je nach Größe der Collection werden neue Extents als Speicherbereiche zugewiesen oder freigegeben. An dieser Stelle kommt eine interessante Eigenschaft von MongoDB zu tragen - das sog. *Padding*. Dabei reserviert MongoDB bei der Speicherung eines Dokuments grundsätzlich mehr Speicherplatz als benötigt wird, um ein zukünftiges Anwachsen des Dokuments abzufangen. Dadurch sollen ressourcenintensive interne Verschiebungen von Dokumenten im Speicher möglichst

[24] Vgl. MongoDB (o. A.), 2023b. Die Referenz bezieht sich auf den ganzen Absatz.
[25] Vgl. Trelle, 2014, Kap. 2.4.1 – 2.4.2, S. xxiii f. Die Referenz bezieht sich auf den ganzen Absatz.
[26] Vgl. Trelle. 2014, Kap. 3.4, S. 4 f. Die Referenz bezieht sich auf den ganzen Absatz.
[27] Vgl. Trelle, 2014, Kap. 3.4, S. 5.
[28] Vgl. MongoDB (o. A.), 2023c. Die Referenz bezieht sich auf die Aussage und das Beispiel.
[29] Vgl. ebda; Trelle, 2014, Kap. 3.4, S. 5. Die Referenz bezieht sich auf den ganzen Absatz und das Beispiel.
[30] Vgl. MongoDB (o. A.), 2023d.

vermieden werden. Der Padding-Faktor ändert sich automatisch in Abhängigkeit der auf Dokumenten durchgeführten Updates.[31]

Dokumente

Nachdem wir nun erfahren haben wie wir eine Datenbank, eine Collection und unser erstes Dokument anlegen können, wollen wir uns intensiver mit den Dokumenten als Herzstück unserer Dokumenten-Datenbanken beschäftigen.

Wir kennen bereits die interne und externe Repräsentation der Dokumente in MongoDB und betrachten die Dokumente nun inhaltlich. Zunächst gilt, dass jedes Dokument einen Primärschlüssel mit dem Datenfeld *_id* erhält. Diese ID dient der eindeutigen Identifizierung eines Dokuments und kann entweder explizit durch den Anwender definiert werden oder sie wird durch die Datenbank vergeben, sollte keine explizite Angabe vorliegen. Die Literatur empfiehlt grundsätzlich die ID durch die Datenbank erzeugen zu lassen, um möglichen Kollisionen vorzubeugen.[32]

In unserem zuvor erstellten Dokument *test* haben wir keine ID erzeugt. Wenn wir das Dokument ausgeben lassen, sehen wir, dass eine ID von der Datenbank erzeugt wurde:

db.uebung.findOne()[33]

Eine automatisch vergebene ID beinhaltet immer einen Zeitstempel. Mit dem Befehl db.uebung.findOne()._id.getTimestamp() können wir uns diesen ausgeben lassen. Wird das Datenfeld *_id* als Index genutzt, beeinflusst der Zeitstempel auch die Sortierung der Dokumente in der Datenbank.[34]

In MongoDB sind Schreiboperationen auf die gespeicherten Dokumente *atomar*. Dies bedeutet, dass keine Operation nur in Teilen durchgeführt wird! Entweder kann die Datenbank die Operation vollständig durchführen oder sie wird überhaupt nicht durchgeführt.[35]

Nachfolgend werden wir kennenlernen, wie wir Operationen auf Dokumenten ausführen können. Diese folgenden in MongoDB den unter *CRUD* bekannten Operationen[36].

Insert

Um Dokumente in eine MongoDB-Datenbank einzufügen, wird der Befehl *insert()* verwendet. Dabei kommt das sog. *Fire-and-Forget-Prinzip* zum Tragen, was zur Folge hat, dass der MongoDB-Server nicht ohne explizite Rückfrage den Status der letzten Operation zurückgibt. Der Vorteil dieses Prinzips liegt in einer höheren Abfrageperformance.[37]

Um ein Dokument in unsere zuvor angelegte Collection einzufügen könnten wir beispielsweise folgendermaßen vorgehen[38]:

db.uebung.insert({name: “Mustermann“, vorname: “Max“, geb: “01.01.2000“})

[31] Vgl. Trelle, 2014, Kap 3.4, S. 6 ff. Die Referenz bezieht sich auf den ganzen Absatz.
[32] Vgl. Trelle, 2014, Kap. 3.5.4, S. 10g. Die Referenz bezieht sich auf den ganzen Absatz.
[33] Vgl. MongoDB (o. A.), 2023e.
[34] Vgl. Trelle, 2014, Kap. 3.5.3, S. 10f. Die Referenz bezieht sich auf den ganzen Absatz.
[35] Vgl. Trelle, 2014, Kap. 3.5, S. 10b. Die Referenz bezieht sich auf den ganzen Absatz.
[36] Vgl. Trelle, 2014, Kap. 7, S. 171.
[37] Vgl. Trelle, 2014, Kap. 7.1, S. 171. Die Referenz bezieht sich auf den ganzen Absatz.
[38] Vgl. MongoDB (o. A.), 2023c. Die Referenz bezieht sich auf das Beispiel.

Das Dokument ist nun Teil der Collection *uebung* in der Datenbank *uebung_fdbt* und hat automatisch eine ID von der Datenbank erhalten.

Über einen sogenannten *bulk insert* können auch mehrere Dokumente in eine Collection eingefügt werden, indem die Dokumente der insert-Methode in einem Array übergeben werden. Die Felder können im Sinne der uns bereits bekannten Schemalosigkeit völlig unterschiedlich sein, wobei auch hier automatisch IDs für die Dokumente von der Datenbank erzeugt werden[39]:

```
db.uebung.insert([
        {name: “THM“, typ: “Hochschule“},
        {vereinsname: “FC Testlauf“, ort: „Friedberg“}
])
```

Update

Mit Hilfe der Methode *update()* können Manipulationen an bereits in der Datenbank bestehenden Dokumenten vorgenommen werden. Dabei ist wichtig, dass bei bestehenden Dokumenten zunächst einzelne Datenfelder verändert werden können. Andererseits können auch ganze Dokumente mit Hilfe der Methode ersetzt werden.[40]

Zunächst ist festzuhalten, dass der Wert des Datenfeldes *_id* nicht verändert werden darf. Alle anderen Felder können mittels Angabe eines Operators geändert werden ohne, dass ein Dokument vollständig ersetzt wird. Durch diese sog. partielle Änderung wird gleichzeitig Netzwerkverkehr eingespart, da nur die zu verändernden Daten gesendet werden.[41]

Wir betrachten folgende Operatoren: *$set, $unset, $rename* und *$push* und werden deren Anwendung jeweils anhand eines Beispiels verdeutlichen. Zunächst soll jedoch ein Überblick darüber gegeben werden, was die Operatoren im Einzelnen bewirken[42]:

- **$set:** Setzt einen neuen Wert in einem oder mehreren Feldern, wobei nicht vorhandene Felder angelegt werden

  ```
  db.uebung.update({vorname: ‘Max‘}, {$set: {vorname: ‘Maxi‘}})
  ```

- **$unset:** Löscht ein oder mehr vorhandene Felder. Um ein Feld zu löschen, wird dessen Name als Schlüssel zusammen mit dem Wert 1 übergeben

  ```
  db.uebung.update({vorname: ‘Maxi‘}, {$unset: {vorname:1}})
  ```

- **$rename:** Benennt ein oder mehr Felder mit einem anderen Namen

  ```
  db.uebung.update({name:  ‘Mustermann‘}, {$rename: {name: ‘Familienname‘}})
  ```

[39] Vgl. MongoDB (o. A.), 2023f. Die Referenz bezieht sich auf den ganzen Absatz und das Beispiel.
[40] Vgl. MongoDB (o. A.), 2023g. Die Referenz bezieht sich auf den ganzen Absatz.
[41] Vgl. Trelle, 2014, Kap. 7.2-7.2.1, S. 176 f. Die Referenz bezieht sich auf den ganzen Absatz.
[42] Vgl. Trelle, 2014, Kap. 7.2.2, S. 177 ff. Die Referenz bezieht sich auf den Absatz, die Aufzählung und die Beispiele.

- **$push:** Wird für Arrays verwendet und fügt einem Array ein Element an. Auch hier werden nicht vorhandene Felder automatisch angelegt

  db.uebung.update({vereinsname: 'FC Testlauf'}, {$push: {punkte:1}})

Remove

Wollen wir einzelne Dokumente oder auch eine gesamte Collection löschen, so können wir dies mit Hilfe der *remove()*- und *drop()*-Methoden durchführen. *Remove()* löscht ein oder mehrere spezifische Dokumente bzw. alle, sofern kein Filter gesetzt wird und *drop()* löscht die gesamte Collection, was einen zeitlichen Mehrwert gegenüber dem löschen aller Dokumente in der Collection mit Hilfe von *remove()* mit sich bringt[43]:

- Löschen einzelner Dokumente:

  db.uebung.remove({_id: 5})

- Löschen einer gesamten Collection

  db.test.drop()

Spezielle Operationen- Upsert

Soll ein Dokument verändert werden, welches möglicherweise noch nicht in der Datenbank persistiert ist, so kann mit *upsert,* einer Zusammensetzung aus der *insert()*- und *upsert()*-Methode, ein Dokument eingefügt werden bevor es verändert wird. *Upsert* ist dabei eine Option innerhalb der *update()*-Methode, welche standardmäßig deaktiviert ist.[44]

Folgende Beispiel-Anfrage könnten wir dazu an MongoDB senden[45]:

db.uebung.update({name:"Olaf"}, {$set: {beruf:"Bundeskanzler"}}, {upsert:true})

Indizes

Ein weiteres Konzept von MongoDB sind *Indizes*. Diese erhöhen die Performance von Abfragen auf eine Collection. Ohne Index müssten stets alle Dokumente einer Collection nacheinander untersucht werden, um eine Abfrage zu erfüllen. Der Index speichert dazu die Werte der als Index gewählten Datenfelder. Grundsätzlich dient das uns bereits bekannte *_id*-Feld als Standard-Index und sorgt damit auch für die Eindeutigkeit der Dokumenten-IDs. Daneben können vielfältige weitere Indizes erstellt werden. Beispielsweise auf einem spezifischen Datenfeld.[46]

Ein Index wird intern als B-Baum abgespeichert und ermöglicht ebenfalls eine performantere Sortierung von Dokumenten. Ein Nachteil der Indizierung sind zeitintensivere

[43] Vgl. Trelle, 2014, Kap. 7.4, S. 6a ff. Die Referenz bezieht sich auf den ganzen Absatz, die Aufzählung und die Beispiele.
[44] Vgl. ebda., Kap. 7.2.3, S. 2. Die Referenz bezieht sich auf den ganzen Absatz.
[45] Vgl. code_blooded7, 2022. Die Referenz bezieht sich auf das Beispiel.
[46] Vgl. MongoDB (o. A.), 2023h. Die Referenz bezieht sich auf den ganzen Absatz.

Schreiboperationen auf der Datenbank. Wir haben die Möglichkeit Indizes auf Datenfeldern von Dokumenten, aber auch auf Arrays und Datenfelder von eingebetteten Dokumenten anzulegen. Wird ein Wert von 1 übergeben, wird der Index aufsteigend sortiert, mit einem Wert von -1 absteigend. Folgende Befehle werden dafür benötigt[47]:

- Index auf einem Datenfeld *name*, aufsteigend sortiert:

  db.uebung.ensureIndex({name:1})

- Index auf einem Datenfeld eines eingebetteten Dokuments (Punktoperator (!)):

  db.uebung.ensureIndex({unterDoc.datenfeld: -1})

- Index auf Arrays, wir benötigen keinen besonderen Befehl und behandeln das Array als Datenfeld:

  db.uebung.ensureIndex ({array:1})

- Rückgabe der Indizes einer Collection:
  db.uebung.getIndexes()

Eingebettete Dokumente

Grundsätzlich ist es mit MongoDB möglich Referenzen zwischen Dokumenten herzustellen und damit normalisierte Daten zu erzeugen. Dazu wird das *_id*-Feld eines Dokuments in einem anderen Dokument als Referenz platziert. MongoDB empfiehlt dieses Vorgehen in bestimmten Szenarien. Beispielsweise, wenn eine Einbettung zu starken Redundanzen führen würde oder sehr komplexe Datenbeziehungen modelliert werden sollen.[48]

In Kontrast dazu, bietet MongoDB die Möglichkeit, Dokumente ineinander einzubetten und so ein de-normalisiertes Datenmodell zu erzeugen. Dabei werden sog. *sub-documents* innerhalb eines Dokuments platziert. Dies entspricht gewissermaßen einer Hierarchie innerhalb des Dokuments. Ein Vorteil dabei ist, dass die Anzahl an Queries und Update-Operationen reduziert werden kann. MongoDB empfiehlt dieses Vorgehen immer dann, wenn wenn eine 1:1-Beziehung oder eine 1:n-Beziehung zwischen Dokumenten bzw. Entitäten bestehen. [49]

Um mit eingebetteten Dokumenten zu interagieren, verwenden wir den Punktoperator. Auf diese Weise können wir uns in der Hierarchie des Dokuments „entlang hangeln“[50]:

db.uebung.find({“doc.field_of_sub_doc“: “value“})

Queries und Projektionen

Wir haben bereits gelernt, wie wir Dokumente finden können. Wir sind dabei aber eher pragmatisch vorgegangen und haben uns aufgrund der überschaubaren Datenmenge auch damit

[47] Vgl. Trelle, 2014, Kap. 6.4-6.4.3, S. 6t ff. Die Referenz bezieht sich auf den ganzen Absatz und die Beispiele.
[48] Vgl. MongoDB (o. A.), 2023i. Die Referenz bezieht sich auf den ganzen Absatz.
[49] Vgl. ebda. Die Referenz bezieht sich auf den ganzen Absatz.
[50] Vgl. MongoDB (o. A.), 2023j. Die Referenz bezieht sich auf den ganzen Absatz und das Beispiel.

zufriedengegeben, ganze Dokumente als Rückgabe zu erhalten. Werden Dokumente komplexer oder der Datenbestand größer und wollen wir beispielsweise nur bestimmte Datenfelder erhalten, so benötigen wir auch konkretere Suchmechanismen. MongoDB erlaubt dazu die Erstellung von Suchanfragen, welche Filterkriterien und unterschiedliche Bedingungen entgegennehmen können.[51]

Solche Bedingungen werden über Operatoren realisiert, von denen wir nun eine Auswahl kennenlernen werden. Grundsätzlich nutzen wir die uns bereits bekannte *find()*-Methode, übergeben dieser jedoch unsere Operatoren. Wir können dabei auch mehrere Operatoren einsetzen, indem wir die Query um logische Operatoren ergänzen.[52]

Folgende Operatoren wollen wir uns näher ansehen[53]:

- **$ne:** Gibt die Werte zurück, welche **ungleich** des definierten Wertes sind
- **$in:** Erhält ein Array und gibt die Werte zurück, welche gleich den Werten in dem Array sind
- **$gt:** Gibt die Werte **größer** des Werts zurück
- **$lt:** Gibt die Werte **kleiner** des Werts zurück
- **$and:** Logischer Operator, **beide** Bedingungen müssen zutreffen
- **$or:** Logischer Operator, **eine** der Bedingungen muss zutreffen
- **$eq:** Gibt Werte **gleich** des Werts zurück

Wir wollen dies anhand von zwei Beispielen veranschaulichen[54]:

- **$or:**

  db.uebung.find({$or: [{name: “THM“}, {name: “Olaf“}]})

- **$in:**

  db.uebung.find({name: { $in: [“Olaf“, “THM“] } })

- **$gt:**

  db.uebung.find({ income: { $gt: 20000} })

Sofern wir nur bestimmte Datenfelder von angefragten Dokumenten erhalten wollen, verwenden wir eine sog. *Projektion*. In dieser Projektion definieren wir die Datenfelder, welche wir erhalten oder nicht erhalten wollen. Wir können mit Projektionen somit Datenfelder einschließen oder ausschließen. Wichtig ist, dass nicht beides gleichzeitig möglich ist. Wir entscheiden uns also entweder für das Ausschließen von Datenfeldern der zurückgegebenen Dokumente oder für das Einschließen von Datenfeldern. Lediglich das *_id*-Feld, welches grundsätzlich ungefragt zurückgegeben wird kann auch bei einer einschließenden Projektion ausgeschlossen werden. Eine Projektion wird realisiert, indem der Query die Schlüsselfelder gemeinsam mit dem Wert 1 (Einschließen) oder dem Wert 0 (Ausschließen) übergeben werden.

[51] Vgl. Papiernik, 2021. Die Referenz bezieht sich auf den ganzen Absatz.
[52] Vgl. ebda. Die Referenz bezieht sich auf den ganzen Absatz.
[53] Vgl. MongoDB (o. A.), 2023k. Die Referenz bezieht sich auf die ganze Aufzählung.
[54] Vgl. Papiernik, 2021. Die Referenz bezieht sich auf die ganze Aufzählung und die Beispiele.

Folgendes Beispiel veranschaulicht das Vorgehen für eine Projektion, welche das Datenfeld *typ* einschließt und das Datenfeld *_id* ausschließt[55]:

```
db.uebung.find( {name: "THM"}, {_id:0, typ:1} )
```

Auch für Datumsangaben können wir Operatoren verwenden, so bspw. den Operator *$year*, welcher es uns ermöglicht Datenfelder in Hinblick auf Jahre abzufragen. Unter zusätzlicher Nutzung des *$expr*-Operators könnten wir dann z. B. folgende Abfrage durchführen, welche das Datenfeld *Familienname* aller Dokumente zurückgibt, welche im Datenfeld *geb* das Jahr 2000 enthalten[56]:

```
db.uebung.find ( {$expr: { $eq: [{$year: "$geb"}, 2000] } }, {_id:0, Familienname:1})
```

[55] Vgl. Papiernik (2021). Die Referenz bezieht sich auf den ganzen Absatz und das Beispiel.
[56] Vgl. AQSA, 2023. Die Referenz bezieht sich auf den ganzen Absatz und das Beispiel.

Literatur

AQSA (2023): *MongoDB Query Using Dates.* [Online] verfügbar unter: https://sparkbyexamples.com/mongodb/mongodb-query-using-dates/, Zugriff 04.06.2023.

Brueck (2015): *Konsistenz*. In: *Wirtschaftsinformatik Wiki – Kewee*. [Online] verfügbar unter: https://wi-wiki.de/doku.php?id=bigdata:konsistenz, Zugriff am 29.05.2023.

Code_blooded7 (2022): *Upsert in MongoDB*. [Online] verfügbar unter: https://www.geeksforgeeks.org/upsert-in-mongodb/, Zugriff am 02.06.2023.

Ian (2021): *MongoDB Date() Method*. [Online] verfügbar unter: https://database.guide/mongodb-date-method/, Zugriff am 04.06.2023.

Inray Industriesoftware GmbH (o. A.) (2023): *Was ist JSON? Praxisnahes Basiswissen.* [Online] verfügbar unter: https://www.opc-router.de/was-ist-json/, Zugriff am 29.05.2023.

JSON.org (o. A.) (o. J.): *Einführung in JSON.* [Online] verfügbar unter: https://www.json.org/json-de.html, Zugriff am 29.05.2023.

MongoDB (o. A.) (2023a): *JSON and BSON.* [Online] verfügbar unter: https://www.mongodb.com/json-and-bson, Zugriff am 29.05.2023.

MongoDB (o. A.) (2023b): *Was ist MongoDB?* [Online] verfügbar unter: https://www.mongodb.com/de-de/what-is-mongodb, Zugriff am 30.05.2023.

MongoDB (o. A.) (2023c): *Datenbank in MongoDB erstellen*. [Online] verfügbar unter: https://www.mongodb.com/de-de/basics/create-database, Zugriff am 02.06.2023.

MongoDB (o. A.) (2023d): *db.collection.countDocuments()*. [Online] verfügbar unter: https://www.mongodb.com/docs/manual/reference/method/db.collection.countDocuments/, Zugriff am 02.06.2023.

MongoDB (o. A.) (2023e): *db.collection.findOne()*. [Online] verfügbar unter: https://www.mongodb.com/docs/manual/reference/method/db.collection.findOne/, Zugriff am 02.06.2023.

MongoDB (o. A.) (2023f): *db.collection.insert()*. [Online] verfügbar unter: https://www.mongodb.com/docs/manual/reference/method/db.collection.insert/, Zugriff am 30.05.2023.

MongoDB (o. A.) (2023g): *db.collection.update()*. [Online] verfügbar unter: https://www.mongodb.com/docs/manual/reference/method/db.collection.update/, Zugriff am 30.05.2023.

MongoDB (o. A.) (2023h): *Indexes*. [Online] verfügbar unter: https://www.mongodb.com/docs/manual/indexes/#footnote-b-tree, Zugriff am 30.05.2023.

MongoDB (o. A.) (2023i): *Data Model Design*. [Online] verfügbar unter: https://www.mongodb.com/docs/manual/core/data-model-design/, Zugriff am 02.06.2023.

MongoDB (o. A.) (2023j): *Query on Embedded/Nested Documents.* [Online] verfügbar unter: https://www.mongodb.com/docs/manual/tutorial/query-embedded-documents/#std-label-read-operations-embedded-documents, Zugriff am 03.06.2023.

MongoDB (o. A.) (2023k): *Query and Projection Operators.* [Online] verfügbar unter: https://www.mongodb.com/docs/manual/reference/operator/query/, Zugriff am 03.06.2023.

Oracle (o. A.) (2023a): *Was ist eine relationale Datenbank (RDMBS)?* [Online] verfügbar unter: https://www.oracle.com/de/database/what-is-a-relational-database/, Zugriff am 28.05.2023.

Oracle (o. A.) (2023b): *Was bedeutet NoSQL?* [Online] verfügbar unter: https://www.oracle.com/de/database/nosql/what-is-nosql/, Zugriff am 28.05.2023.

Oracle (o. A.) (2023c): *JSON definiert*. [Online] verfügbar unter: https://www.oracle.com/de/database/what-is-json/, Zugriff am 29.05.2023.

Papiernik, Mateusz (2021): *How To Create Queries in MongoDB.* [Online] verfügbar unter: https://www.digitalocean.com/community/tutorials/how-to-create-queries-in-mongodb, Zugriff am 03.06.2023.

Roe, Charles (2012): *ACID vs. BASE: The Shifting pH of Database Transaction Processing.* [Online] verfügbar unter: https://www.dataversity.net/acid-vs-base-the-shifting-ph-of-database-transaction-processing/, Zugriff am 29.05.2023.

Saake, Gunter; Sattler Kai-Uwe; Heuer, Andreas (2018): *Datenbanken – Konzepte und Sprachen.* 6. Auflage. *MITP.* ISBN: 978-3-95845-777-5.

Trelle, Tobias (2014): *MongoDB: Der praktische Einstieg.* 1. Auflage. *dpunkt.verlag.* ISBN (ePub): 978-3-86491-534-5.